

Dispositivos Conectados

Experimentação com mecanismos de segurança no ESP32-C6: eFUSES, AES e HMAC

2025/2026

Authors: Paulo C. Bartolomeu

1 Objetivos

- Introdução ao uso de eFuses no ESP32-C6
- Cifra/decifra usando os algoritmos AES-ECB e AES-CBC
- Geração e verificação de *tickets* através do algoritmo HMAC

2 Introdução

Os eFuses no ESP32-C6 desempenham um papel central na segurança de dispositivos IoT, oferecendo uma forma segura e permanente de armazenar informação sensível. Num cenário cada vez mais exposto a ataques físicos e lógicos, especialmente em dispositivos conectados à internet, proteger dados críticos como chaves criptográficas, configurações de *boot* e políticas de acesso torna-se um requisito fundamental. Os eFuses viabilizam essa proteção ao permitir gravações irreversíveis de bits de configuração e dados, garantindo que informações essenciais não possam ser alteradas ou lidas indevidamente após sua definição.

A arquitetura do ESP32-C6 inclui um conjunto de blocos de eFuses divididos em áreas específicas com propósitos distintos. Entre os usos mais relevantes está o armazenamento de chaves simétricas de 256 bits, utilizadas em operações criptográficas de *hardware* como HMAC, AES-XTS e criptografia de *flash*. Esses blocos de chave são denominados KEY_BLOCK_x (onde _x representa o número do bloco, como KEY_BLOCK0, KEY_BLOCK1, etc.) e cada um possui capacidade de armazenar uma chave de 256 bits de forma permanente.

A segurança desses blocos é reforçada por bits auxiliares de controlo, como os WR_DIS (*Write Disable*) e RD_DIS (*Read Disable*), que, uma vez ativados, impedem a regravação ou leitura da chave, respectivamente. Essa lógica de proteção é irreversível: após a gravação de uma chave e ativação dos bits de proteção, o conteúdo torna-se imutável e inacessível externamente. Isso permite que as chaves armazenadas sirvam como raízes de confiança para sistemas de autenticação, criptografia de armazenamento e *boot* seguro, sem risco de exposição.

Além das chaves, os eFuses controlam outras funcionalidades críticas, como a habilitação do *secure boot*, ativação da criptografia de *flash* por *hardware*, restrições de acesso a JTAG, e definição de versões mínimas de *firmware* (*anti-rollback*).

Contudo, manipulação dos eFuses exige cuidado, já que qualquer escrita é definitiva e irreversível.

Por isso, o desenvolvimento seguro com ESP32-C6 costuma envolver testes iniciais em dispositivos de desenvolvimento com chaves temporárias, seguidos de uma etapa de provisionamento seguro em produção.

Nesta aula iremos explorar a utilização de eFuses em combinação com algoritmos de cifra AES e de *digest* HMAC.

3 Trabalho Prático

O trabalho prático desta aula desenvolve-se com base na [API do eFuse Manager](#), juntamente com a biblioteca [Mbed TLS](#), particularmente os módulos de [AES](#) e [Hashing](#) e HMAC.

3.1 eFuses

Nesta secção iremos escrever uma chave pre-definida num bloco dos eFuses. O objetivo é simular o cenário em que dispositivos IoT são provisionados uma chave única que apenas o fabricante conhece. No exemplo seguinte iremos usá-la para cifrar informação que só o fabricante ou o próprio dispositivo IoT poderá decifrar.

Antes de prosseguir é necessário consultar o estado atual dos eFuses da placa ESP32-C6, o que pode ser feito no Terminal. Para o efeito, execute o comando `control + shift + P` e deverá observar a paleta de comandos da Figura 1. Aí deverá pesquisar a opção ESP-IDF: Open ESP-IDF Terminal visível na referida figura.

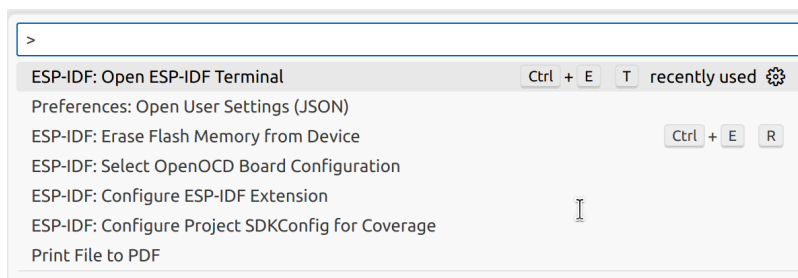


Figura 1: Terminal para executar comandos

Em seguida execute no terminal o seguinte comando, onde `<ESP32-C6 serial port>` poderá ser, por exemplo, `/dev/ttyUSB0`, caso a porta série da sua placa tenha sido montada nesta porta.

```
~$ idf.py -p <ESP32-C6 serial port> efuse-summary
```

O resultado deste comando permite ler e exibir o estado atual dos fusíveis programáveis do dispositivo ESP32-C6 (neste caso, conectado à porta `/dev/ttyUSB0`), mostrando um resumo completo de todos os blocos de eFuse, incluindo configurações de segurança (como chaves criptográficas, MAC address, proteções de leitura/gravação, etc.).

```
Config fuses:
WR_DIS (BLOCK0)          Disable programming of individual eFuses      = 0 R/W
  (0x00000000)
RD_DIS (BLOCK0)          Disable reading from BLOCK4-10             = 0 R/W
  (0b00000000)
...
MAC (BLOCK1)             MAC address
```

<code>= 58:8c:81:3b:c0:bc (OK) R/W</code>		
<code>MAC_EXT (BLOCK1)</code>	Stores the extended bits of MAC address	<code>= ff:fe</code>
<code>(OK) R/W</code>		
<code>CUSTOM_MAC (BLOCK3)</code>	Custom MAC	
<code>= 00:00:00:00:00:00 (OK) R/W</code>		
<code>MAC_EUI64 (BLOCK1)</code>	<code>calc MAC_EUI64 = MAC[0]:MAC[1]:MAC[2]:MAC_EXT[0]:M</code>	
<code>= 58:8c:81:ff:fe:3b:c0:bc (OK) R/W</code>	<code>AC_EXT[1]:MAC[3]:MAC[4]:MAC[5]</code>	
<code>...</code>		
<code>Security fuses:</code>		
<code>DIS_DOWNLOAD_ICACHE (BLOCK0)</code>	Represents whether icache is disabled or enabled	<code>i = False</code>
<code>R/W (0b0)</code>	<code>n Download mode. 1: disabled. 0: enabled</code>	
<code>...</code>		
<code>SECURE_BOOT_EN (BLOCK0)</code>	Represents whether secure boot is enabled or disabled	<code>= False</code>
<code>R/W (0b0)</code>	<code>led. 1: enabled. 0: disabled</code>	
<code>...</code>		
<code>BLOCK_KEY4 (BLOCK8)</code>		
<code>Purpose: USER</code>		
<code>Key4 or user data</code>	<code>s</code>	
<code>= 00 R/W</code>		
<code>BLOCK_KEY5 (BLOCK9)</code>		
<code>Purpose: USER</code>		
<code>Key5 or user data</code>		
<code>= 00 R/W</code>		

ALERTA!: Os blocos de chaves eFuse **são de escrita única (irreversível)**. Uma vez escrito, o bloco não pode ser reescrito. Face a isto, deve proceder com grande cautela para os seguintes passos.

De modo a criar uma chave comum num bloco eFuse, iremos usar um programa que possui uma função para o efeito.

QUESTÃO 1: Descompacte o ficheiro `efuse.zip` e abra o projeto correspondente no VSCode. Analise o ficheiro `main.c` e indique o que faz a função `write_key_to_efuse_dev_only()`. Na vida de um dispositivo IoT uma variante desta função pode ser usada para fazer a programação de fábrica de chaves simétricas a instalar no dispositivo.

QUESTÃO 2: Na função `app_main(void)` descomente essa função, faça um *clean* global, compile o código e descarregue o binário resultante para a placa. Execute o programa e deverá observar que a chave pré-configurada foi programada no bloco `BLOCK_KEY5`. Isso pode ser confirmado usando novamente o comando `idf.py -p /dev/ttyUSB0 efuse-summary` no Terminal, tal como se apresenta em seguida:

```
~$ idf.py -p <ESP32-C6 serial port> efuse-summary
...
BLOCK_KEY4 (BLOCK8)
  Purpose: USER
    Key4 or user data
    = 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 R/W
BLOCK_KEY5 (BLOCK9)
  Purpose: USER
    Key5 or user data
    = 00 01 02 03 04 05 06 07 08 09 0a 0b 0c 0d 0e 0f 10 11 12 13 14 15 16 17 18 19 1a 1b 1c 1d 1e 1f R/W
```

Isto significa que a chave foi efetivamente instalada na memória do dispositivo IoT, ficando disponível agora para ser usada em diferentes operações criptográficas. Agora que esta operação está concluída, **comente a invocação da função `write_key_to_efuse_dev_only()` na função `app_main(void)`.**

3.2 Cifra e decifra usando o algoritmo AES-256 com chave guardada num eFuse

O próximo passo é usar a chave armazenada para cifrar texto em claro. Esse texto cifrado pode posteriormente ser comunicado ou armazenado localmente, sendo que apenas quem possua a chave simétrica será capaz de revelar o texto em claro.

QUESTÃO 3: Analise a função `example_aes_encrypt_decrypt_with_efuse_key()` e identifique o resultado da sua execução. Descomente a respectiva invocação na função `app_main(void)`, compile e descarregue para a placa o novo binário. Deverá observar na consola a seguinte informação:

```
I (246) main_task: Started on CPU0
I (246) main_task: Calling app_main()
I (246) AES_EFUSE: eFuse key read successfully.
I (246) AES_EFUSE: Plaintext: ConnectedDevices
I (246) AES_EFUSE: b7 da 85 6b 8e 7f 06 3b 1a 3a d3 65 aa aa cd df
I (256) AES_EFUSE: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
I (256) AES_EFUSE: Decrypted: ConnectedDevices
I (266) main_task: Returned from app_main()
```

QUESTÃO 4: O que acontece se a mensagem possuir uma dimensão superior à do exemplo? Implemente uma solução para cifrar mensagens de tamanho arbitrário.

QUESTÃO 5: O método de cifra empregue nas questões anteriores é o AES-256-ECB. Altere o método de cifra para AES-256-CBC e repita o teste. Qual o resultado da execução do programa?

3.3 Geração de HMAC (Hash-based Message Authentication Code)

A chave guardada pode ser utilizada para outros fins que não a simples cifra de informação. Existindo uma chave única atribuída por um fabricante a cada dispositivo IoT, ela pode ser usada como um *token* de identidade que só um determinado equipamento possui. Neste sentido pode ser usada para gerar um *ticket/digest* associado a uma mensagem de dimensão arbitrária.

QUESTÃO 6: Descompacte o ficheiro `hmac.zip` e abra o projeto correspondente no VSCode. Analise o ficheiro `main.c` e indique o que faz a função `compute_hmac_sha256(...)`. Faça o clean global do projeto, compile e descarregue o binário para a placa. Deverá observar o seguinte resultado:

```
I (253) main_task: Started on CPU0
I (253) main_task: Calling app_main()
I (253) HMAC_SHA256: HMAC-SHA256 output:
I (253) HMAC_SHA256: 9f b6 1e f2 17 3c 7f 0e d0 da 96 e0 6a 79 57 de
I (253) HMAC_SHA256: cf f8 20 75 ef 56 a7 59 78 0c 4e 33 d5 35 8e b8
I (263) main_task: Returned from app_main()
```

QUESTÃO 7: Confirme o resultado do algoritmo HMAC no site <https://emn178.github.io/online-tools/sha256.html>. Use as seguintes configurações para verificar o programa:

```
key(hex): 000102030405060708090A0B0C0D0E0F101112131415161718191A1B1C1D1E1F
input: Connected Devices 2025/2026
output: 9fb61ef2173c7f0ed0da96e06a7957decff82075ef56a759780c4e33d5358eb8
```

QUESTÃO 8: Como poderá ser usado este mecanismo para auditar registos de um dispositivo IoT, i.e., que os registos provêm efetivamente de um determinado equipamento IoT e não de outro do mesmo fabricante?

4 Acknowledgements

Originally authored by [Paulo C. Bartolomeu](#)